



TECHNISCHES ONBOARDING

MhM verstehen

Architektur, Datenbank, Abläufe, Patterns und reale Codebeispiele - erklärt für Junior Developer

Analysierter Stand: 15.09.2026

Projekt: MhM.UI / .NET 10 / Blazor Interactive Server

Basis: Quellcode, EF-Core-Modell und Migrationen; Build erfolgreich (0 Warnungen, 0 Fehler)



So benutzt du dieses Handbuch

Dieses Dokument ist keine vollständige Zeile-für-Zeile-Wiedergabe. Es vermittelt zuerst ein belastbares mentales Modell und führt dann zu den Dateien, in denen du Details nachlesen kannst. Lies Kapitel 1 bis 4 am Stück. Danach kannst du die Use Cases in Kapitel 6 einzeln nachvollziehen und parallel im Code debuggen.

Kurz gesagt

MhM ist ein modularer Monolith: eine deploybare ASP.NET-Core-Anwendung enthält UI, Geschäftslogik und Datenzugriff. Die Trennung erfolgt über Ordner, Komponenten und Services - nicht über mehrere Prozesse oder Projekte.

Inhaltsverzeichnis

So benutzt du dieses Handbuch	2
Inhaltsverzeichnis	3
1. Das Projekt in einem Satz	5
1.1 Die wichtigsten Rollen	5
1.2 Die Kernreise eines Auftrags	5
2. Technologiestapel und Laufzeitmodell	5
2.1 Wichtige NuGet-Pakete	6
2.2 Was beim Seitenaufruf passiert	6
3. Projekt- und Dateilandkarte	6
3.1 Razor und Code-behind	7
4. Start der Anwendung: Program.cs	7
4.1 Service-Registrierung	7
4.2 Konfiguration nach Umgebung	7
4.3 Middleware-Reihenfolge	8
5. Datenbank und Domänenmodell	8
5.1 Warum es IdentityUser und AppUser gibt	8
5.2 Tabellen nach Verantwortungsbereich	9
5.3 Fremdschlüssel und DeleteBehavior	9
5.4 Datenbank-Constraints als Sicherheitsnetz	9
5.5 Migrationen	9
6. Zentrale Use Cases Schritt für Schritt	9
6.1 Registrierung und Anmeldung	9
6.2 Auftrag anlegen oder bearbeiten	10
6.3 Suchen und Filtern	10
6.4 Bewerbung	10
6.5 Matching und Vergabe	10
6.6 Unveränderliche Vereinbarung	11
6.7 Chat und Benachrichtigungen	11
6.8 Abschluss und Bewertungen	11
6.9 Problem, Blockierung und Moderation	11
7. Verwendete Patterns und Prinzipien	12
7.1 Warum kein Repository pro Entity?	12
7.2 AsNoTracking und Include	12
7.3 Records	12
8. Sicherheit und Datenschutz	13
8.1 Bildvalidierung als gutes Beispiel	13
8.2 Beobachtete Risiken und Verbesserungen	13
9. Was gut gelöst ist - und was wachsen sollte	13

9.1 Stärken	13
9.2 Technische Schulden / nächste sinnvolle Schritte	14
9.3 Konkretes Zielbild ohne Overengineering	14
10. Praktische Code-Lesetechnik	14
10.1 Einen Use Case rückwärts verfolgen	14
10.2 LINQ lesen	14
10.3 Async/Await	15
10.4 Fehlerbehandlung	15
11. Lokales Arbeiten und Debugging	15
11.1 Typischer Ablauf	15
11.2 Gute Breakpoints	15
11.3 SQL sichtbar machen	15
12. Empfohlener Einarbeitungsplan	16
12.1 Gute erste Tickets	16
13. Glossar	17
14. Dateiindex: Wo suche ich was?	18
14.1 Quellenbasis und Abgrenzung	18
Merksatz zum Abschluss	18

1. Das Projekt in einem Satz

MhM - vermutlich 'Menschen helfen Menschen' - ist eine regionale Vermittlungsplattform. Auftraggeber veröffentlichen kleine Aufträge, Helfer bewerben sich, ein Auftraggeber vergibt den Auftrag, beide stimmen sich im Chat ab und bewerten sich nach Abschluss. Profile, Blockierungen, Meldungen, Moderation und Benachrichtigungen bilden die Vertrauensschicht.

1.1 Die wichtigsten Rollen

Rolle	Im Code	Darf hauptsächlich
Privatperson	UserRole.Privatperson	Aufträge erstellen, vergeben und Helfer bewerten
Helfer	UserRole.Helfer + HelperProfile	Aufträge suchen, sich bewerben, Arbeit durchführen
Unternehmen	UserRole.Unternehmen	Im Datenmodell vorgesehen; noch wenig eigener Ablauf
Administrator	Identity-Rolle Admin	Nutzer sperren, Meldungen bearbeiten, Aufträge moderieren

Wichtig ist die Trennung zwischen einer fachlichen Rolle in `AppUser.Rolle` und der technischen Berechtigungsrolle `Admin` aus ASP.NET Core Identity. Ein Admin ist also nicht einfach ein weiterer Wert im fachlichen Enum.

1.2 Die Kernreise eines Auftrags

Normaler Pfad oben; Abbruch/Moderation oder Konfliktpfad unten.

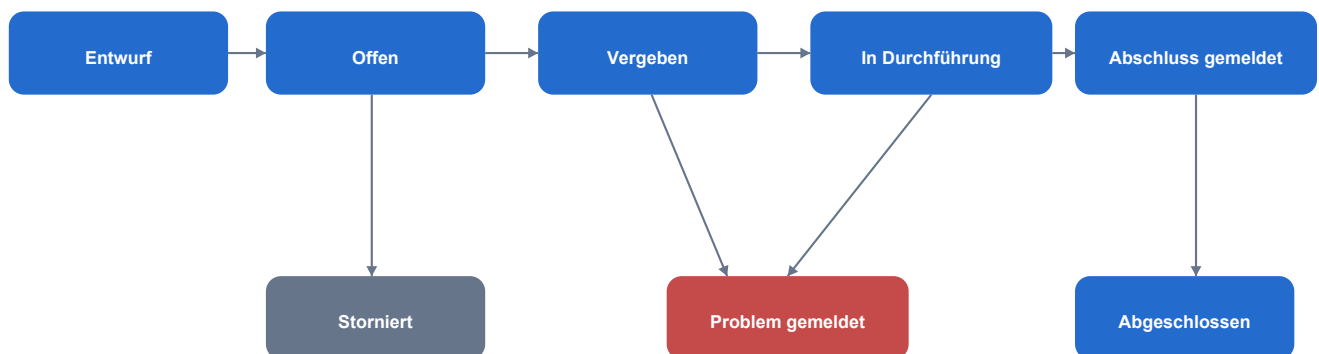


Abbildung 1: Die im Code verwendete Auftrags-Statusmaschine.

2. Technologiestapel und Laufzeitmodell

Die Anwendung zielt auf .NET 10 und verwendet ASP.NET Core mit Blazor Interactive Server. 'Server' bedeutet: C#-Komponenten laufen auf dem Server. Der Browser hält eine interaktive Verbindung und empfängt UI-Differenzen. JavaScript wird nur für browsernahe Aufgaben wie Login-Fetch, Geolocation, Theme, Popover und Benachrichtigungen genutzt.



Abhängigkeiten laufen nach rechts; Ereignisse und Daten fließen in beide Richtungen.

Abbildung 2: Vereinfachte Schichten und technische Abhängigkeiten.

2.1 Wichtige NuGet-Pakete

Paket	Aufgabe	Warum relevant
Microsoft.EntityFrameworkCore.SqlServer	ORM und SQL-Server-Treiber	LINQ-Abfragen, Migrationen und Transaktionen
Identity.EntityFrameworkCore	Benutzer, Passwörter, Rollen, Cookies	Sichere Standardmechanismen statt eigener Passwortlogik
Azure.Identity	Managed Identity / Token	Produktionszugriff auf Azure SQL ohne Passwort in der App
Blazor.Bootstrap	UI-Komponenten	Zusätzliche Darstellungskomponenten

2.2 Was beim Seitenaufruf passiert

- Der Browser fordert eine URL an, zum Beispiel /auftraege/{id}.
- ASP.NET Core durchläuft Middleware: HTTPS, Lokalisierung, Antiforgery, Rate Limit, Authentifizierung und Autorisierung.
- Der Router findet die Razor-Komponente anhand von @page.
- Blazor erzeugt die Komponenteninstanz und füllt [Parameter] sowie [Inject].
- Ein Lifecycle-Hook wie OnParametersSetAsync lädt Daten über DbContextFactory.
- StateHasChanged beziehungsweise das Ende des Hooks führt zu einem neuen Render-Diff im Browser.

Fachbegriff: Dependency Injection

Abhängigkeiten werden nicht mit new in jeder Klasse erzeugt. Der Container kennt Zuordnungen wie IMatchingService -> MatchingService und reicht passende Objekte ein. Das senkt Kopplung und erleichtert Tests.

3. Projekt- und Dateilandkarte

Die Solution enthält derzeit ein einziges Webprojekt. Das ist für eine kompakte Anwendung übersichtlich, bedeutet aber auch, dass Präsentation, Use-Case-Logik und Persistenz teilweise eng beieinander liegen.

Pfad	Verantwortung	Startpunkte
MhM.UI/Program.cs	Composition Root, Middleware, Minimal APIs	Service-Registrierung und HTTP-Pipeline
Components/App.razor	HTML-Shell, Styles, Blazor-Skripte	Wurzel des Dokuments
Components/Routes.razor	Routing + AuthorizeRouteView	Seitenwahl und Zugriffsschutz
Components/Pages	Seiten und UI-nahe Abläufe	Auftraege, AuftragDetail, Mein, Admin
Components/Layout	Navigation und Notification Center	MainLayout

Pfad	Verantwortung	Startpunkte
Components/Shared	Wiederverwendbare UI-Bausteine	Avatar, Carousel, Rating
Data/Models	Fachliche Entities und Enums	MhMModels.cs
Data/MhMDbContext.cs	EF-Core-Mapping	Tabellen, Beziehungen, Constraints
Services	Wiederverwendbare Geschäftslogik	Matching, Notifications, Images, Geocoding
Migrations	Versionsgeschichte des Schemas	Up/Down + ModelSnapshot
wwwroot	CSS und Browser-JavaScript	app.css, site.js

3.1 Razor und Code-behind

Viele Seiten bestehen aus `X.razor` und `X.razor.cs`. Beide Teile werden durch `partial class` zu einer Klasse zusammengefügt. Die Razor-Datei beschreibt das Markup; die C#-Datei enthält Zustand, Lifecycle und Ereignisbehandlung.

```
// AuftragDetail.razor
@page "/auftraege/{ListingId:guid}"

// AuftragDetail.razor.cs
public partial class AuftragDetail
{
    [Parameter] public Guid ListingId { get; set; }
    protected override async Task OnParametersSetAsync() { ... }
}
```

Lesart: Der Router parst die URL, validiert den `guid`-Constraint und setzt `ListingId`. Danach lädt der Lifecycle-Hook das passende Entity.

Pattern: Code-behind

UI und Logik bleiben dieselbe Komponente, sind aber in zwei Dateien getrennt. Das verbessert Lesbarkeit, ist jedoch keine echte Architekturschicht. Direkte Datenbanklogik im Code-behind bleibt weiterhin UI-gekoppelt.

4. Start der Anwendung: Program.cs

`Program.cs` ist der Composition Root: Hier werden Abhängigkeiten zusammengesetzt, Konfiguration gelesen, Middleware angeordnet und Endpunkte registriert.

4.1 Service-Registrierung

```
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

builder.Services.AddDbContextFactory<MhMDbContext>(...,
    ServiceLifetime.Singleton);

builder.Services.AddScoped<INotificationService, NotificationService>();
builder.Services.AddScoped<IMatchingService, MatchingService>();
```

Scoped bedeutet normalerweise eine Instanz pro HTTP-Request beziehungsweise Circuit-Scope. Die `DbContextFactory` erzeugt dagegen bewusst für jede Operation einen neuen Context. Das ist in lang lebenden Blazor-Server-Komponenten wichtig, weil ein einzelner Context nicht thread-safe ist und sonst sehr lange Entities tracken würde.

4.2 Konfiguration nach Umgebung

Development und Test verwenden die Connection String `MhM-dev` für LocalDB. Andere Umgebungen verwenden `MhM` für Azure SQL. Beginnt die Zeichenfolge mit `Server=tcp:`, wird `DefaultAzureCredential` über einen Connection-Interceptor eingesetzt. Zusätzlich aktiviert EF Core Retry-Logik gegen vorübergehende Netzwerkfehler.

4.3 Middleware-Reihenfolge

Reihenfolge	Middleware	Wirkung
1	Exception Handler / HSTS	Produktionsfehler und HTTPS-Härtung
2	Localization	de-DE standardmäßig, en-US alternativ
3	Status Code Pages / HTTPS	404-Seite und Umleitung
4	Antiforgery / RateLimiter	CSRF-Schutz und Login-Drosselung
5	Authentication / Authorization	Cookie lesen, Principal erzeugen, Regeln prüfen
6	Endpoints / Razor Components	Minimal APIs, Bilder und UI

Best Practice

Reihenfolge ist Verhalten. Autorisierung kann nur funktionieren, wenn Authentifizierung vorher den Benutzer bestimmt hat. Rate Limiting schützt den Login mit zehn Versuchen pro IP und Minute zusätzlich zum Identity-Lockout.

5. Datenbank und Domänenmodell

EF Core bildet C#-Entities auf SQL-Server-Tabellen ab. `DbSet<T>` ist der Einstieg für Abfragen. `OnModelCreating` definiert Schlüssel, Indizes, Beziehungen, Längen, Dezimalpräzision und Löscherhalten.

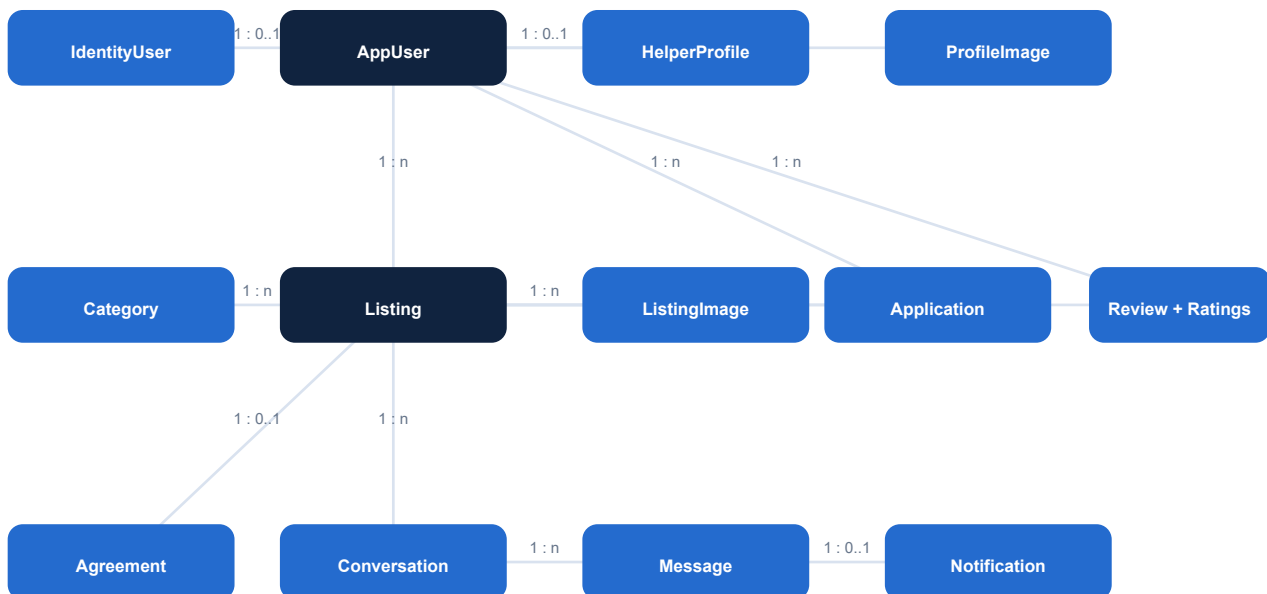


Abbildung 3: Zentrale Beziehungen; Identity-Standardtabellen sind zusammengefasst.

5.1 Warum es IdentityUser und AppUser gibt

`ApplicationIdentityUser` ist das technische Konto: Loginname, Passwort-Hash, Lockout, Security Stamp und Rollen. `AppUser` ist das fachliche Profil: Anzeigenname, Ort, Rolle, Verifikationen und Beziehungen zu Aufträgen. Verbunden werden beide über `AppUser.IdentityUserId`.

Trade-off

Die Trennung schützt die Fachdomäne vor vielen Identity-Details und erlaubt Demo-AppUsers ohne Login. Sie erzeugt aber Konsistenzarbeit: Registrierung und Profiländerung müssen zwei Datensätze synchron halten.

5.2 Tabellen nach Verantwortungsbereich

Bereich	Tabellen	Wichtige Invarianten
Identität	AspNetUsers, Roles, Claims ...	Unique Email, Lockout, Security Stamp
Profil	AppUsers, HelperProfiles, ProfileImages	Username unique; je Profil maximal ein Bild/Helferprofil
Marktplatz	Categories, Listings, ListingImages, ListingApplications	Bewerbung pro Auftrag/Helfer unique
Vertrag	AssignmentAgreements	Je Auftrag maximal eine unveränderliche Vereinbarung
Kommunikation	Conversations, Messages, UserNotifications	Conversation-Kombination unique; Notification pro Message unique
Vertrauen	Reviews, ReviewCategoryRatings, UserBlocks	Review unique; Sterne 1..5; kein Self-Block
Moderation	ContentReports, AdminAuditLogs	Statusindex und nachvollziehbare Admin-Aktionen

5.3 Fremdschlüssel und DeleteBehavior

Cascade löscht abhängige Daten automatisch, etwa ListingImages beim Löschen eines Listings. Restrict verhindert das Löschen referenzierter Benutzer und schützt Historie. SetNull löst eine optionale Verbindung, etwa Conversation einer Notification. Diese Regeln sind fachliche Entscheidungen, nicht nur Datenbanktechnik.

5.4 Datenbank-Constraints als Sicherheitsnetz

```
applications.HasIndex(x => new { x.ListingId, x.ApplicantId })
    .IsUnique();

reviews.ToTable(t => t.HasCheckConstraint(
    "CK_Reviews_Stars", "[Stars] >= 1 AND [Stars] <= 5"));
```

Die UI prüft dieselben Regeln für gute Fehlermeldungen. Der Unique Index und die Check Constraint verhindern aber auch Fehler bei parallelen Requests oder zukünftigen Importskripten. Das nennt man Defense in Depth.

5.5 Migrationen

Migrationen sind versionierte Schemaänderungen. Die Historie zeigt die Entwicklung: Grundschemata, Identity, Bilder, Bewerbungsstatus, Bewertungskategorien, Chat/Notifications, öffentliche Profile, Profilbilder, Trust/Moderation, unveränderliche Vereinbarungen und eindeutige Reviews. Beim Start ruft die App Database.MigrateAsync() auf.

Produktionshinweis

Automatische Migration beim App-Start ist bequem, kann bei mehreren Instanzen oder großen Tabellen aber riskant sein. Reifere Deployments führen Migrationen oft als kontrollierten Release-Schritt aus.

6. Zentrale Use Cases Schritt für Schritt

6.1 Registrierung und Anmeldung

- Register.razor bindet Eingaben an ein validiertes InputModel.
- UserManager.CreateAsync erzeugt das Identity-Konto und hasht das Passwort.
- Danach wird ein AppUser mit derselben IdentityUserId gespeichert.
- Scheitert Schritt 3, wird das Identity-Konto kompensierend wieder gelöscht.
- Browser-JavaScript ruft /account/logon auf; SignInManager erstellt das Auth-Cookie.

```
var result = await UserManager.CreateAsync(identityUser, model.Password);
if (!result.Succeeded) { ... }

try {
    db.AppUsers.Add(appUser);
    await db.SaveChangesAsync();
}
catch {
    await UserManager.DeleteAsync(identityUser); // Kompensation
}
```

Das ist eine kleine Saga/kompensierende Aktion: Da Identity und Profil nicht gemeinsam in derselben expliziten Operation gespeichert werden, wird ein schon erfolgreicher Schritt rückgängig gemacht, wenn der zweite scheitert.

6.2 Auftrag anlegen oder bearbeiten

- [Authorize] schützt beide Routen /auftraege/neu und /bearbeiten.
- Die Komponente löst den eingeloggten Identity-Benutzer zum AppUser auf.
- DataAnnotations validieren das Formular; zusätzliche Regel prüft BudgetMin <= BudgetMax.
- Der Geocoding-Service übersetzt PLZ/Ort optional in Koordinaten.
- Beim Bearbeiten werden Eigentümer und zulässiger Status erneut serverseitig geprüft.
- Bilder werden anhand echter Bytes validiert, Metadaten werden entfernt und Daten in SQL gespeichert.

Warum doppelte Autorisierungsprüfung?

[Authorize] sagt nur: angemeldet. Erst die Entity-Prüfung `listing.RequesterId == currentUserId` beantwortet: Darf genau dieser Benutzer genau diesen Auftrag ändern? Das ist Object-Level Authorization.

6.3 Suchen und Filtern

Auftraege baut eine IQueryable-Abfrage schrittweise auf. Kategorie, Text, Ort, Vergütung und Budget laufen in SQL. Für Geo-Suche werden danach alle vorgefilterten Treffer geladen und die Haversine-Distanz im Speicher berechnet. 'Mehr laden' ist Client-seitiges Batching der bereits geladenen Liste, keine echte Datenbank-Paginierung.

```
var filteredListings = await listingQuery
    .OrderByDescending(x => x.CreatedUtc)
    .ToListAsync();

filteredItems = filteredListings
    .Select(x => new { Listing = x,
        DistanceKm = CalculateDistanceKm(...) })
    .Where(x => x.DistanceKm <= radius)
    .ToListAsync();
```

Skalierungsgrenze

Bei wenigen Daten ist das verständlich. Bei vielen Aufträgen sollten Geo-Filter und Pagination in die Datenbank wandern, zum Beispiel mit SQL Spatial, Skip/Take oder Cursor-Pagination.

6.4 Bewerbung

Nur AppUsers mit Rolle Helfer dürfen sich auf einen offenen fremden Auftrag bewerben. Vor dem Insert prüft der Code Status, Eigentümer, vorhandene Bewerbung und wechselseitige Blockierung. Der Unique Index auf (ListingId, ApplicantId) bleibt die letzte Garantie gegen Race Conditions.

6.5 Matching und Vergabe

Das Ranking berechnet fünf Teilwerte auf einer Skala von 0 bis 100. Kompatible Vergütung und Radius sind harte Filter; die restlichen Kriterien werden gewichtet.

Kriterium	Gewicht	Berechnung / Default
Distanz	35 %	Linear bis zum Radius; unbekannt = 50
Bewertung	25 %	Durchschnitt / 5; ohne Daten = 3,5 Sterne
Preis	20 %	Im Budget = 100; außerhalb abgestuft

Kriterium	Gewicht	Berechnung / Default
Zuverlässigkeit	15 %	abgeschlossene / beteiligte Aufträge; unbekannt = 50
Verifizierung	5 %	verifiziert = 100, sonst 0

```
var total =
    distanceScore * 0.35 +
    ratingScore   * 0.25 +
    priceScore    * 0.20 +
    reliabilityScore * 0.15 +
    verificationScore * 0.05;
```

Die Gewichte sind eine Policy. Sie sollten später durch Produktanforderungen, Messdaten, Fairness-Tests und gegebenenfalls Konfiguration abgesichert werden. Aktuell sind sie fest im Code codiert.

Bei der Vergabe führt `MatchingService.AssignListingAsync` in einer retriable Transaction mehrere Änderungen atomar aus: Listing auf Vergeben, Gewinner auf Angenommen, andere Bewerbungen auf Abgelehnt, Agreement erzeugen, Conversation sicherstellen, Systemnachricht speichern. Erst nach Commit wird die separate Notification erzeugt.

6.6 Unveränderliche Vereinbarung

Zum Vergabezeitpunkt kopiert `AssignmentAgreement` Titel, Beschreibung, Preis, Vergütungsart, Termin und Ort. Zusätzlich wird ein SHA-256-Hash aus den Werten erzeugt. Damit bleibt der vereinbarte Stand sichtbar, auch wenn das Listing später anders dargestellt würde.

Wichtige Nuance

Ein Hash macht Daten nicht automatisch rechtssicher oder unveränderlich. Die Datenbankrechte und fehlende Update-Pfade sind entscheidend. Außerdem ist die aktuelle String-Zusammensetzung ohne kanonisches, versioniertes Format empfindlich gegenüber Kultur- oder Formatänderungen.

6.7 Chat und Benachrichtigungen

Chat ist nur für Auftraggeber und angenommenen Helfer verfügbar. Nachrichten werden gespeichert; anschließend erzeugt der `NotificationService` eine Benachrichtigung. Die Detailseite pollt alle acht Sekunden, das `MainLayout` Benachrichtigungen alle zehn Sekunden. `DeliveredUtc` und `ReadUtc` bilden Zustellung und Lesen ab.

```
chatRefreshTimer = new PeriodicTimer(TimeSpan.FromSeconds(8));
while (await timer.WaitForNextTickAsync(cancellationToken))
{
    await InvokeAsync(async () => {
        await RefreshChatAsync(cancellationToken);
        StateHasChanged();
    });
}
```

`InvokeAsync` wechselt auf den Blazor-Synchronisationskontext, bevor UI-State verändert wird. `IAsyncDisposable` beendet Timer und `CancellationTokenSource` beim Verlassen der Komponente - ein wichtiges Resource-Lifecycle-Pattern.

6.8 Abschluss und Bewertungen

Der Helfer wechselt Vergeben -> InDurchführung -> AbschlussGemeldet. Der Auftraggeber bestätigt anschließend Abgeschlossen, jedoch nicht vor dem Wunschtermin. Bewertungen sind eindeutig, kategoriebasiert und nach Abgabe nicht mehr durch den Benutzer änderbar. Sie werden erst sichtbar, wenn beide bewertet haben oder 14 Tage vergangen sind. Das ist ein Double-blind Review Window, das taktische Gegenbewertungen reduziert.

6.9 Problem, Blockierung und Moderation

Beteiligte können einen Problemfall eröffnen; Vertrag und Chat bleiben zur Klärung erhalten. Blockierungen verhindern neue Kontakte, brechen bestehende Vereinbarungen aber nicht heimlich ab. Reports haben Status; Admin-Aktionen schreiben `AdminAuditLog` und können Moderationsbenachrichtigungen auslösen.

7. Verwendete Patterns und Prinzipien

Pattern / Prinzip	Fundstelle	Nutzen	Grenze
Modularer Monolith	ein Webprojekt	einfach zu bauen/deployen	Grenzen nur durch Disziplin
Dependency Injection	Program.cs, [Inject]	austauschbare Abhängigkeiten	viele Pages greifen trotzdem direkt auf DB zu
Repository-artiger DbContext	DbSet + LINQ	Unit of Work und Abfragen	kein eigenes Repository - bewusst okay
Service Layer	Matching/Notification/Image	komplexe, wiederverwendbare Regeln zentral	nicht alle Use Cases ausgelagert
DTO / Projection	MatchCandidate, NotificationListItem	UI erhält nur benötigte Daten	Pages laden teils volle Entities
State Machine	ListingStatus	erlaubte Übergänge nachvollziehbar	Regeln auf mehrere Methoden verteilt
Optimistic Concurrency	Identity ConcurrencyStamp	Konflikterkennung bei Konten	fachliche Entities besitzen keinen RowVersion-Token
Defense in Depth	Validation + Constraints	Schutz trotz paralleler Zugriffe	DbUpdateException sollte gezielt behandelt werden
Polling	Chat/Layout Timer	einfach und robust	mehr DB-Last und Verzögerung als SignalR-Push
Compensation	Registrierung	räumt Teilfehler auf	keine echte verteilte Transaktion

7.1 Warum kein Repository pro Entity?

EF Core liefert bereits Repository- und Unit-of-Work-Eigenschaften: DbSet sammelt Abfragen und SaveChanges schreibt eine Arbeitseinheit. Ein zusätzliches generisches Repository würde hier oft nur Methoden wie GetAll oder Add weiterreichen. Sinnvoller wären Use-Case-Services, wenn komplexe Page-Logik ausgelagert werden soll.

7.2 AsNoTracking und Include

AsNoTracking() ist für reine Leseabfragen gedacht und reduziert Speicher/Tracking-Aufwand. Include lädt Navigation Properties. Zu viele Includes können große kartesische Resultsets erzeugen; deshalb ist in Azure die SplitQuery-Strategie konfiguriert.

```
var conversation = await db.Conversations
    .AsNoTracking()
    .Include(x => x.Requester)
    .Include(x => x.Helper)
    .FirstOrDefaultAsync(...);
```

7.3 Records

record eignet sich für unveränderlich gedachte Datencontainer mit Wertgleichheit, zum Beispiel MatchCandidate. Entities bleiben Klassen, weil EF Core Identität, Navigationen und Mutation trackt.

8. Sicherheit und Datenschutz

Schutz	Implementierung	Bedrohung
Passwort-Policy + Hashing	ASP.NET Core Identity	schwache oder offengelegte Passwörter
Lockout	5 Fehlversuche / 5 Minuten	Account-Bruteforce
Rate Limit	10 Loginversuche/IP/Minute	automatisiertes Credential Stuffing
HttpOnly Cookie	MhM.Auth	direkter JS-Zugriff auf Session-Cookie
Antiforgery	Middleware + Logout-Validierung	CSRF
Role/Entity Checks	[Authorize] + Owner/Participant	IDOR / fremde Datensätze
Bild-Sniffing	PNG/JPEG-Signatur + Dimensionen	gefälschte Upload-Typen und Dekompressionsrisiken
Metadatenentfernung	EXIF/XMP/IPTC/Text-Chunks	Standort- und Geräte-Leaks
Managed Identity	Azure credential interceptor	DB-Passwort in Produktion
Audit Log	AdminAuditLogs	nicht nachvollziehbare Moderation

8.1 Bildvalidierung als gutes Beispiel

```
if (!ProfileImageSecurity.TryValidate(bytes,
    out var detectedContentType, out var validationError) ||
    !ProfileImageSecurity.TryRemoveMetadata(
        bytes, detectedContentType, out var sanitized))
{
    return (false, validationError);
}
```

Der Code vertraut nicht nur dem vom Browser gemeldeten MIME-Type. Er prüft Datei-Signatur und Header, begrenzt Dateigröße, Kantenlänge und Pixelzahl und speichert eine bereinigte Kopie. Das ist ein starkes Beispiel für serverseitige Validierung.

8.2 Beobachtete Risiken und Verbesserungen

- Die Login-Antwort ist absichtlich generisch - gut gegen Benutzer-Ermittlung.
- SecurePolicy.SameAsRequest ist lokal bequem; in Produktion wäre Always klarer, sofern ausschließlich HTTPS verwendet wird.
- Bilddaten in SQL vereinfachen Konsistenz, erhöhen aber DB-Größe und Backup-Last. Object Storage wäre später denkbar.
- Content Security Policy ist im sichtbaren Startup nicht konfiguriert; externe Bootstrap-Icons kommen von einem CDN.
- Die Anwendung protokolliert wenig sicherheitsrelevante Ereignisse außerhalb des AdminAuditLog.
- Das Entwicklungs-Admin-Passwort admin/admin wird nur außerhalb Production geseedet, bleibt aber in gemeinsam genutzten Testumgebungen riskant.

9. Was gut gelöst ist - und was wachsen sollte

9.1 Stärken

- Sauberer erfolgreicher Build ohne Warnungen.
- Serverseitige Berechtigungsprüfungen an sensiblen Operationen.
- Datenbank-Constraints für Eindeutigkeit und Wertebereiche.
- Transaktion und ExecutionStrategy bei der komplexen Vergabe.
- Unveränderlicher Vertragssnapshot und Double-blind Bewertungen.

- Sichere Bildprüfung mit Entfernung von Metadaten.
- Kurzlebige DbContexts, Cancellation bei Polling und korrekte Dispose-Pfade.
- Managed Identity und Retry für Azure SQL.

9.2 Technische Schulden / nächste sinnvolle Schritte

Priorität	Beobachtung	Empfehlung
Hoch	Keine Testprojekte im Repository	Unit-Tests für Status/Scoring, Integrations-Tests für EF-Constraints und Auth
Hoch	AuftragDetail ca. 1.100 Zeilen über Partial-Dateien	Use Cases in Services teilen: Application, Review, Trust, Chat
Hoch	Notification nach Vergabe erst nach Commit	Transactional Outbox gegen verlorene Notification bei Prozessabbruch
Mittel	Geo-Suche und Load More im Speicher	DB-seitige Spatial Query und echte Pagination
Mittel	Keine RowVersion auf Listing	Optimistic Concurrency für parallele Vergabe/Bearbeitung
Mittel	Polling alle 8/10 Sekunden	SignalR Push oder gebündelte Abfrage
Mittel	Einige Regeln doppelt in Pages	Policy/Use-Case-Services als Single Source of Truth
Niedrig	Kommentare wie NEU / alte auskommentierte Konfiguration	Bereinigen, Architekturentscheidungen in ADRs dokumentieren

Keine pauschale Kritik

Für die aktuelle Größe ist ein modularer Monolith vernünftig. Ziel ist nicht, möglichst viele Schichten zu erzeugen, sondern Geschäftsregeln testbar und einmalig zu platzieren.

9.3 Konkretes Zielbild ohne Overengineering

```

Components/Pages      // dünne UI-Orchestrierung
Application/
  Listings/           // Create, Apply, Assign, Complete
  Reviews/            // Submit + Visibility Policy
  Moderation/         // Report, Block, Admin actions
Domain/
  ListingWorkflow.cs  // erlaubte Zustandsübergänge
Infrastructure/
  Persistence/        // DbContext, Migrations
  Geocoding/          // Google Adapter
  Notifications/      // DB + optional Outbox/SignalR

```

Dieses Zielbild kann weiterhin ein einziges Projekt und eine einzige Deployment-Einheit bleiben. Die Ordnergrenzen dienen Testbarkeit und Orientierung.

10. Praktische Code-Lesetechnik

10.1 Einen Use Case rückwärts verfolgen

- Beginne bei der sichtbaren Schaltfläche in der .razor-Datei.
- Suche den gebundenen Handler, zum Beispiel AssignAsync.
- Notiere [Inject]-Abhängigkeiten und folge dem Interface zur Implementierung.
- Suche alle Änderungen an Entity.Status oder SaveChangesAsync.
- Prüfe OnModelCreating auf Constraints und DeleteBehavior.
- Prüfe, welche Rolle/Eigentümerschaft vor dem Schreiben validiert wird.
- Setze Breakpoints vor Query, vor SaveChanges und nach Navigation.

10.2 LINQ lesen

```
var acceptedExists = await db.ListingApplications
    .AnyAsync(x => x.ListingId == listingId &&
        x.Status == ListingApplicationStatus.Angenommen,
        cancellationToken);
```

Lies von links nach rechts: Tabelle ListingApplications; existiert mindestens eine Zeile; deren ListingId gleich der gesuchten ID ist; und deren Status Angenommen ist? AnyAsync wird typischerweise zu SQL EXISTS.

10.3 Async/Await

Datenbank, HTTP und Browser-Interop sind I/O. await blockiert keinen Thread, während auf das Ergebnis gewartet wird. Ein CancellationToken ermöglicht Abbruch, etwa wenn die Komponente verschwindet. await using entsorgt den DbContext auch bei Exceptions.

10.4 Fehlerbehandlung

Die UI übersetzt erwartbare Fehler meist in deutschsprachige Meldungen. Services werfen bei Regelverletzungen InvalidOperationException oder UnauthorizedAccessException. Für langfristige Wartbarkeit wären eigene Exception-Typen oder ein Ergebnisobjekt wie Result<T> aussagekräftiger.

11. Lokales Arbeiten und Debugging

11.1 Typischer Ablauf

```
dotnet restore MhM.slnx
dotnet build MhM.slnx
dotnet run --project MhM.UI/MhM.UI.csproj
```

In Development wird LocalDB über MhM-dev verwendet. Beim Start laufen Migrationen sowie Demo- und Admin-Seeding. Der Google-Maps-Key gehört in User Secrets und nicht in appsettings oder Git.

```
dotnet user-secrets set "GoogleMaps:ApiKey" "DEIN_KEY" --project MhM.UI/MhM.UI.csproj
```

11.2 Gute Breakpoints

Frage	Breakpoint
Warum darf ich nicht bearbeiten?	Auftrag.OnParametersSetAsync / SaveAsync bei Owner-Prüfung
Warum erscheint ein Bewerber nicht?	MatchingService.GetTopCandidatesForListingAsync im foreach
Warum ist ein Statuswechsel abgelehnt?	MatchingService.TransitionForHelperAsync
Warum sehe ich den Chat nicht?	AuftragDetail.LoadChatStateAsync
Warum ist eine Bewertung unsichtbar?	LoadSubmittedReviewsAsync bei releaseReached
Warum fehlt eine Notification?	NotificationService.Create... und RefreshNotificationsAsync

11.3 SQL sichtbar machen

In Program.cs ist EF-Logging derzeit absichtlich stumm. Für lokale Diagnose kann zeitweise LogTo(Console.WriteLine, ...) aktiviert werden. EnableSensitiveDataLogging() nur lokal und kurz verwenden, weil Parameter personenbezogene Daten enthalten können.

Build-Stand

Der analysierte Stand wurde mit dotnet build MhM.slnx geprüft: erfolgreich, 0 Warnungen, 0 Fehler.

12. Empfohlener Einarbeitungsplan

Tag	Leseziel	Praktische Übung
1	Program.cs, App.razor, Routes.razor	App starten; Request und Circuit im Debugger beobachten
2	MhMMModels.cs, MhMDbContext.cs	ER-Skizze selbst nachzeichnen; drei LINQ-Queries formulieren
3	Auftrag + Auftraege	Auftrag anlegen, filtern, editieren; SaveAsync verfolgen
4	MatchingService + Vergabe	Scores von Hand berechnen; Statusübergänge testen
5	Chat + NotificationService	Message -> Notification -> Delivered/Read verfolgen
6	Reviews, Blocks, Reports, Admin	Missbrauchsfälle und Berechtigungsgrenzen notieren
7	Migrationen + Sicherheit	kleine Migration in Übungsbranch; ersten Integrationstest schreiben

12.1 Gute erste Tickets

- Unit-Tests für CalculatePriceScore und Statusübergänge ermöglichen (Methoden/Policy extrahieren).
- Einen Integrationstest für den Unique Index von Bewerbungen erstellen.
- AsNoTracking bei weiteren reinen Leseabfragen prüfen.
- Doppelte Apply-Logik aus Auftraege und AuftragDetail in einen Service ziehen.
- Ein ADR zur Entscheidung SQL-Bildspeicherung verfassen.
- Geo-Pagination messen, bevor optimiert wird.

13. Glossar

Begriff	Einfache Erklärung
Entity	C#-Objekt mit eigener Identität, das meist einer Datenbankzeile entspricht.
ORM	Übersetzt zwischen Objekten/LINQ und relationalem SQL; hier Entity Framework Core.
DbContext	Arbeitseinheit für Queries, Change Tracking und SaveChanges.
Migration	Versionierte, reproduzierbare Änderung am Datenbankschema.
Navigation Property	C#-Verweis auf verbundene Entities, zum Beispiel Listing.Requester.
Foreign Key	Spalte, die auf den Primärschlüssel einer anderen Tabelle zeigt.
Invariant	Regel, die immer gelten muss, etwa genau eine Bewerbung je Helfer/Auftrag.
DTO	Datencontainer für eine Grenze; enthält nur die benötigten Werte.
Dependency Injection	Abhängigkeiten werden von außen bereitgestellt.
Middleware	Baustein in der HTTP-Pipeline vor dem eigentlichen Endpunkt.
Authentication	Wer bist du? In MhM über Identity-Cookie.
Authorization	Was darfst du? Rolle, Eigentümer oder Chat-Teilnehmer.
Circuit	Langlebige Blazor-Server-Sitzung zwischen Browser und Server.
Lifecycle Hook	Framework-Methode wie OnParametersSetAsync zu einem definierten Zeitpunkt.
Projection	LINQ-Select in eine kleinere Form statt kompletter Entities.
Transaction	Mehrere DB-Änderungen gelten gemeinsam oder gar nicht.
Execution Strategy	Wiederholt eine Operation bei kurzzeitigen SQL-/Netzwerkfehlern.
Race Condition	Ergebnis hängt vom Timing paralleler Operationen ab.
Optimistic Concurrency	Erkennt, dass jemand einen Datensatz seit dem Lesen geändert hat.
Idempotenz	Wiederholung erzeugt keinen zusätzlichen unerwünschten Effekt.
Polling	Client fragt regelmäßig nach neuen Daten statt Push zu empfangen.
SignalR	ASP.NET-Technik für Echtzeit-Push; Blazor Server nutzt sie intern ebenfalls.
Managed Identity	Azure-Identität für Dienste ohne fest gespeichertes Passwort.
Hash	Einweg-Fingerabdruck von Daten; Änderung führt zu anderem Ergebnis.
Composition Root	Zentraler Ort, an dem Implementierungen zusammengesetzt werden.
Modularer Monolith	Eine Deployment-Einheit mit bewusst getrennten internen Modulen.

14. Dateiindex: Wo suche ich was?

Wenn du verstehen willst ...	Datei(en)
wie die App startet	MhM.UI/Program.cs
wie Seiten geroutet werden	Components/Routes.razor und @page-Direktiven
wie die Shell aussieht	Components/App.razor, Layout/MainLayout.*
welche fachlichen Daten existieren	Data/Models/MhMModels.cs
wie Tabellen/Beziehungen aussehen	Data/MhMDbContext.cs, Migrations/ModelSnapshot
wie Konten gespeichert werden	ApplicationIdentityUser.cs, Account/Register.*
wie Aufträge erstellt werden	Pages/Auftrag.razor und .razor.cs
wie Suche/Bewerbung läuft	Pages/Auftraege.* und AuftragDetail.*
wie Ranking/Vergabe läuft	Services/MatchingService.cs, Pages/AuftragVergabe.*
wie Chat funktioniert	Pages/AuftragDetail.Chat.cs
wie Benachrichtigungen funktionieren	Services/INotificationService.cs, MainLayout.razor.cs
wie Uploads abgesichert sind	Services/ProfileImageSecurity.cs, IListingImageService.cs
wie Geocoding funktioniert	Services/GoogleGeocodingService.cs
wie Bewertungen/Trust laufen	Pages/AuftragDetail.razor.cs, Models/ReviewSubmission.cs
wie Administration läuft	Pages/Admin.* und Data/AdminSeed.cs
wie Übersetzungen funktionieren	Localization/UiLocalizer.cs, CultureSwitcher.razor

14.1 Quellenbasis und Abgrenzung

Analysiert wurden alle produktiven C#- und Razor-Dateien, das EF-Core-Mapping, die Migrationen, Projekt- und App-Konfiguration sowie JavaScript/CSS in ihrer architektonischen Rolle. Designer-Dateien der Migrationen wurden für das resultierende Schema herangezogen, aber nicht einzeln erklärt. Im Repository ist kein Testprojekt vorhanden. Laufzeitverhalten externer Systeme wie Google Geocoding und Azure SQL wurde aus der Integration abgeleitet, nicht live gegen Produktionsdaten geprüft.

Merksatz zum Abschluss

Die kürzeste mentale Karte

Route -> Razor-Komponente -> Lifecycle/Handler -> Service oder DbContext -> Entity/Constraint -> SaveChanges -> neues UI-Rendering.

Wenn du diesen Pfad für Registrierung, Bewerbung, Vergabe, Chat und Bewertung nachvollziehen kannst, hast du den Kern der Anwendung verstanden.